

Proyecto: LexLP

Analizador Léxico para un Lenguaje de Programación de Alcance Reducido

Curso: Compiladores

Docente: Prof. Nury Y. Arosquipa Yanque

Lenguaje de implementación sugerido: C++

Especificación basada en la lista de expresiones regulares y tokens proporcionada para LP.

1. Descripción del proyecto

Desarrollar un analizador léxico capaz de leer un programa fuente escrito en un lenguaje simplificado denominado LP (Lenguaje de Programación) y transformarlo en una secuencia de tokens. El proyecto implementará también una tabla de símbolos vinculada a los tokens de tipo identificador.

El analizador reconocerá números enteros y decimales, identificadores, constantes de texto, palabras reservadas, comentarios, operadores de asignación, aritméticos, lógicos y relacionales, símbolos especiales y errores léxicos.

La lista de tokens y la tabla de símbolos deberán reproducir computacionalmente el procedimiento realizado previamente de forma manual en clase.

2. Objetivo general

Desarrollar en C++ un analizador léxico para un lenguaje de programación de alcance reducido, utilizando las expresiones regulares definidas para LP, capaz de reconocer tokens, gestionar identificadores mediante una tabla de símbolos y reportar errores léxicos.

3. Alcance del lenguaje LP

LP es un lenguaje imperativo simplificado inspirado en lenguajes como C y Java. En esta etapa del proyecto no se verificará si el programa es sintácticamente o semánticamente correcto; **el objetivo es identificar correctamente sus componentes léxicos.**

```
void main() {  
    int edad = 20;  
    float promedio = 15.5;  
  
    println("Hola mundo");  
  
    if (edad >= 18) {  
        println("Mayor de edad");  
    }  
  
    return;  
}
```

4. Especificación léxica

La implementación deberá utilizar como base la lista de expresiones regulares definida para el lenguaje LP.

4.1 Números e identificadores

Expresion Regular	Token
D = [0-9]	-
L = [a-zA-Z_]	-
D+	NUM_INT
D+\.D+	NUM_DEC
L(L D)*	ID
"*"	TEXT

4.2 Palabras reservadas

Lexema	Token
int	INT
float	FLOAT
char	CHAR
boolean	BOOLEAN
void	VOID
if	IF
else	ELSE
for	FOR
while	WHILE
scanf	SCANF
println	PRINTLN
main	MAIN
return	RETURN

4.3 Operadores

Lexema / ER	Categoría
=	=
+	+
-	-
*	*
/	/
%	%
&&	&&
!	!
> >= < <= != ==	COMP

4.4 Símbolos especiales y comentarios

Lexema / ER	Categoría
(	(
)	)
[	[
]	]
{	{
}	}
,	,
;	;
//.*\n	

Los comentarios deberán ser reconocidos por el analizador. Para el flujo principal de tokens se recomienda reconocerlos y descartarlos, sin incorporarlos a la lista final de tokens.

5. Lista de tokens y tabla de símbolos

Este componente es obligatorio. El analizador deberá producir simultáneamente una lista de tokens y una tabla de símbolos.

5.1 Lista de tokens

La lista de tokens conservará el orden en el que los lexemas aparecen en el programa fuente. Para los tokens que no sean identificadores se devolverá el nombre del token. Cuando el token sea un identificador, deberá incluir como atributo la posición del identificador en la tabla de símbolos.

```
<NUM_INT>
<ID, 0>
<=>
<NUM_INT>
<;>
```

5.2 Tabla de símbolos

Los identificadores serán los únicos elementos almacenados en la tabla de símbolos en esta etapa del proyecto. Cada identificador deberá aparecer una sola vez.

Posición	Identificador
0	edad
1	promedio

Si un identificador vuelve a aparecer en el programa fuente, no se insertará nuevamente. El analizador recuperará la posición existente.

5.3 Ejemplo integrado

```
int edad = 20;
float promedio = edad / 2;
edad = edad + 1;
```

Tabla de símbolos:

Posición	Identificador
0	edad
1	promedio

Lista de tokens:

```
<INT> <ID,0> <=> <NUM_INT> <;>
<FLOAT> <ID,1> <=> <ID,0> </> <NUM_INT> <;>
<ID,0> <=> <ID,0> <+> <NUM_INT> <;>
```

6. Entrada y salida del analizador

6.1 Entrada

El programa recibirá como entrada un archivo de código fuente, por ejemplo **programa.lp**.

```
void main() {
    int edad = 20;
    float promedio = 15.5;

    if (edad >= 18) {
        println("Mayor de edad");
    }
    return;
}
```

6.2 Salidas obligatorias

El analizador deberá mostrar o generar, como mínimo:

- Lista secuencial de tokens.
- Tabla de símbolos.
- Listado de errores léxicos, cuando existan.

Como información de apoyo para depuración, se recomienda registrar línea y columna del lexema reconocido.

7. Manejo de errores léxicos

Todo carácter o secuencia que no pueda asociarse con los tokens especificados deberá reportarse como error léxico. El reporte deberá permitir localizar el error mediante línea y columna.

```
int @edad = 20;
```

Línea	Columna	Lexema	Resultado
1	5	@	ERROR_LEXICO

Los casos límite deberán definirse de acuerdo con la especificación léxica entregada. El proyecto no deberá inventar nuevas construcciones del lenguaje sin documentarlas.

8. Estructuras sugeridas

8.1 Token

```
Token {
    tipo
    atributo    // posición en tabla para ID
    lexema      // opcional para reporte
    linea
    columna
}
```

No es obligatorio almacenar todavía tipo, ámbito, dirección de memoria o valor. Esos atributos pueden incorporarse posteriormente durante el análisis sintáctico y semántico.

9. Organización sugerida del proyecto C++

```
LexLP/
|-- src/
| |-- main.cpp
| |-- Lexer.cpp
| |-- Lexer.h
| |-- Token.cpp
| |-- Token.h
| |-- SymbolTable.cpp
| `-- SymbolTable.h
|-- tests/
| |-- prueba_basica.lp
| |-- prueba_completa.lp
| `-- prueba_errores.lp
|-- output/
| |-- tokens.txt
| |-- tabla_simbolos.txt
| `-- errores.txt
```

10. Desarrollo por fases

Presentar cada fase semanalmente, los martes en el laboratorio, a partir de 15/09

Entrega	Peso	Que se evalua	Fecha
Semana 1	20%	Lectura + NUM_INT + NUM_DEC presentación	15/09
Semana 2	30%	ID + TEXTO + palabras reservadas + tabla de símbolos presentación	22/09
Semana 3	50%	operadores + Tokens completos + Integración + lista de tokens + errores + pruebas + presentación	29/09

11. Casos de prueba obligatorios

Variables y tabla de símbolos

```
int edad = 20;  
float promedio = 15.5;
```

Identificadores repetidos

```
int edad = 20;  
edad = edad + 1;
```

Identificadores válidos

```
int nota1;  
int notaFinal;  
float _promedio;
```

Operadores aritméticos

```
a = b + c;  
x = y * 2;  
z = a % 2;
```

Comparaciones

```
if (a >= b)  
if (a != b)  
if (a == b)
```

Operadores lógicos

```
if (a > 10 && b < 20)  
if (!activo)
```

Comentarios

```
// comentario
int edad = 20;
```

Errores léxicos

```
int @edad;
$variable = 10;
```

12. Ejemplo completo de resultado

```
void main() {
    int edad = 20;
    float promedio = 15.5;
    if (edad >= 18 && edad <= 60) {
        println("Edad valida");
    }
    return;
}
```

Tabla de símbolos esperada:

Posición	Identificador
0	edad
1	promedio

Fragmento de la lista de tokens esperada:

```
<VOID> <MAIN> <( > <{>
<INT> <ID,0> <=> <NUM_INT> <;>
<FLOAT> <ID,1> <=> <NUM_DEC> <;>
<IF> <( <ID,0> <COMP> <NUM_INT> <&&> <ID,0> <COMP> <NUM_INT> <)>
<{> <PRINTLN> <( <TEXTO> <)> <;> <}>
<RETURN> <;> <}>
```

13. Funcionalidades adicionales

- Exportación de resultados a archivos.
- Interfaz gráfica o visualización del proceso de reconocimiento.
- Coloreado sintáctico básico.
- Pruebas automatizadas adicionales.

14. Criterios de evaluación sugeridos

Criterio	Peso
Reconocimiento correcto de los tokens de LP	35%
Lista de tokens	15%
Tabla de símbolos	20%
Manejo de errores léxicos y ubicación	10%
Casos de prueba	10%
Organización, documentación y presentación del código	10%